

Chapitre 3 : Prototype Minimal de Chatbot Embarqué

3.1 Introduction et objectifs

Les deux premiers chapitres de ce mémoire ont établi, respectivement, un état de l’art des modèles de langage embarqués sur smartphone et un protocole de mesure détaillé de leurs performances (débit, latence, consommation énergétique). Ces deux volets restent cependant de nature analytique : ils caractérisent ce qu’un LLM embarqué est capable de faire, sans démontrer comment cette capacité se traduit concrètement dans une application utilisable. Ce troisième chapitre comble cet écart en présentant un prototype logiciel fonctionnel, conçu pour valider que l’inférence locale mesurée au chapitre 2 peut effectivement soutenir une interaction utilisateur réelle, interactive et sans connexion réseau.

L’objectif poursuivi ici n’est donc plus la mesure isolée d’un débit de tokens, mais la démonstration d’un système complet : un utilisateur saisit une requête, le modèle génère une réponse localement, et le résultat est restitué avec des métriques de performance associées. Ce changement de perspective, du benchmark isolé vers l’application de bout en bout, est important pour un mémoire de recherche appliquée : il permet de vérifier que les optimisations techniques du chapitre 2 (quantification Q4_K_M, choix du framework, configuration des threads) ne sont pas seulement valides sur le papier, mais se traduisent en une expérience utilisateur acceptable.

Trois cas d’usage ont été retenus pour cette démonstration, chacun représentatif d’une catégorie d’application couramment envisagée pour les LLMs embarqués dans la littérature (chapitre 1, section 1.2) :

- **Chat interactif (question-réponse)** : conversation libre avec une mémoire de contexte limitée, cas d’usage le plus exigeant en termes de cohérence sur la durée.
- **Résumé de texte** : condensation d’un document fourni par l’utilisateur en points clés, un cas d’usage à faible latence attendue et à forte valeur pratique (notes, articles, messages).
- **Classification de sentiment** : analyse de polarité (positif / négatif / neutre) d’un texte court, illustrant un usage de type classification plutôt que génération libre.

Ces trois cas d’usage ne couvrent pas l’ensemble des applications possibles, mais ils ont été choisis parce qu’ils sollicitent le modèle de façons suffisamment différentes (génération longue avec historique, génération courte guidée par une consigne, classification à sortie contrainte) pour révéler des comportements distincts en termes de latence, de longueur de réponse et de fiabilité. Ce chapitre inclut enfin un module de benchmark automatisé, distinct des scripts de mesure utilisés au chapitre 2 (benchmark_complet.sh et throttling_rigoureux.sh, fondés sur llama-bench), permettant de rejouer un protocole de mesure reproductible directement depuis le même environnement applicatif

que celui utilisé pour l'interaction utilisateur, ce qui garantit que les chiffres rapportés ici décrivent bien le comportement du prototype tel qu'il serait utilisé en pratique, et non un environnement de test isolé.

3.2 Architecture du prototype

Avant de détailler l'implémentation de chaque module, cette section présente les choix de conception structurants du prototype : le langage et l'environnement d'exécution retenus, la pile logicielle utilisée, l'organisation du code en modules, ainsi que le format de prompt commun à l'ensemble des fonctionnalités. Ces choix conditionnent directement les résultats de performance présentés en section 3.8 et méritent donc d'être justifiés individuellement avant d'aborder le code des modules eux-mêmes, présenté à partir de la section 3.3.

3.2.1 Vue d'ensemble

Le choix d'implémentation retenu pour ce prototype est une application en ligne de commande (CLI) écrite en Python, plutôt qu'une application Android complète avec interface graphique. Ce choix mérite d'être justifié : une application Android native (à l'image de celle développée au chapitre 2 pour ML Kit GenAI) démontre l'intégration dans l'écosystème Android, mais elle introduit une couche de complexité supplémentaire (cycle de vie des activités, gestion des permissions, compilation Gradle) qui n'apporte rien à la question centrale de ce chapitre : le modèle peut-il soutenir une conversation multi-tours cohérente et rapide sur l'appareil cible ? Une interface CLI, exécutée directement dans Termux sur le smartphone ou dans un terminal de développement, permet d'isoler cette question sans le bruit expérimental introduit par une interface graphique complète, tout en restant représentative puisque Termux exécute un véritable environnement Linux sur l'appareil Android testé, avec les mêmes contraintes CPU et mémoire.

Le prototype ne repose cependant pas sur un backend d'inférence unique : `chatbot.py` et `benchmark.py`, bien qu'ils partagent le même module `utils.py`, pilotent le modèle de deux façons différentes. `benchmark.py` utilise `llama-cpp-python`, le binding Python officiel du projet `llama.cpp`, dont l'installation est décrite au chapitre 2 (section 1.3), ce qui permet de comparer directement ses résultats à ceux du chapitre 2 (voir section 3.5). `chatbot.py`, en revanche, invoque directement le binaire natif `llama-cli` (compilé au chapitre 2 via `cmake/make`) en sous-processus à chaque tour de parole, plutôt que de charger le modèle une seule fois via les bindings Python.

Ce choix n'est pas arbitraire : il découle d'un problème de compatibilité documenté et récurrent de `llama-cpp-python` sur Termux/Android, rencontré au cours du développement de ce prototype. Sous Python 3.14, `sys.platform` retourne désormais `"android"` (changement récent de CPython), une valeur que le fichier `_ctypes_extensions.py` du

module `llama_cpp` ne reconnaît pas (il ne gère explicitement que `linux`, `freebsd`, `darwin`, `win32` et `emscripten`), ce qui provoque un échec au chargement avec `RuntimeError: Unsupported platform`, y compris lorsque la compilation du wheel elle-même a réussi. Un correctif manuel existe (ajouter la branche `android` au fichier concerné), mais il n’a pas été jugé pertinent de le maintenir pour l’usage interactif : piloter directement le binaire `llama-cli`, déjà compilé et validé au chapitre 2, contourne le problème à la racine tout en restant fiable sur l’ensemble du corpus d’appareils testés. Ce choix illustre un compromis d’ingénierie assez représentatif du développement sur plateforme mobile : une dépendance de haut niveau, plus pratique en théorie (accès direct à l’API Python, pas de découpage de sortie texte), a été écartée au profit d’une solution plus bas niveau mais empiriquement plus robuste dans l’environnement cible. Le code est organisé en trois modules Python distincts, avec une séparation claire des responsabilités :

`prototype-cli/`

```
├─ chatbot.py      # Interface CLI principale : chat, résumé, classification
├─ benchmark.py    # Module de benchmark automatisé
├─ utils.py        # Fonctions utilitaires (métriques, RAM, prompt)
└─ requirements.txt # Dépendances Python
```

Cette organisation en modules suit un principe de conception logicielle courant : `utils.py` centralise les fonctions réutilisables par les deux autres fichiers (construction de prompt, mesure de RAM, structure de métriques), ce qui évite la duplication de code entre l’interface interactive (`chatbot.py`) et le module de mesure automatisée (`benchmark.py`). Un développeur souhaitant, par exemple, changer le format de prompt utilisé n’a qu’un seul endroit à modifier pour que ce changement se répercute sur l’ensemble du prototype.

3.2.2 Pile technologique

Le choix de chaque composante technique répond à une contrainte spécifique du contexte mobile. Le tableau suivant en détaille le rôle exact :

Composante	Technologie	Rôle
Runtime LLM	<code>llama-cpp-python</code> $\geq 0.2.90$ (<code>benchmark.py</code>) ; binaire <code>llama-cli</code> en sous-processus (<code>chatbot.py</code>)	Inférence locale CPU ARM64
Mesures	<code>psutil</code> $\geq 5.9.0$	RAM, CPU, processus
Interface	<code>rich</code> $\geq 13.7.0$	Affichage CLI enrichi
Input	<code>prompt_toolkit</code> $\geq 3.0.43$	Saisie interactive
Export (optionnel, non utilisé)	<code>pandas</code> $\geq 2.0.0$ (commenté)	Analyse résultats JSON (jamais importé dans le code actuel)

Tableau 3.1 : Pile technologique du prototype CLI

`llama-cpp-python` a été retenu pour `benchmark.py` parce qu’il expose une API Python native permettant de récupérer les tokens générés au fur et à mesure (streaming) et d’accéder directement aux métriques internes du moteur, sans les frais généraux d’un

sous-processus lorsque le modèle est chargé une seule fois pour toute la durée d'une session de benchmark. `chatbot.py`, à l'inverse, pilote le binaire `llama-cli` directement, pour les raisons de compatibilité Termux/Android détaillées en section 3.2.1 ; les deux approches coexistent délibérément dans ce prototype plutôt que de chercher une solution unique, chacune étant la mieux adaptée à son cas d'usage.

`psutil` est une bibliothèque standard pour l'inspection système multiplateforme ; elle est utilisée ici pour mesurer la RAM consommée (celle du processus Python pour les fonctions génériques, celle du sous-processus `llama-cli` pour le mode chat, voir section 3.3.2) ainsi que la charge CPU, sans dépendre d'outils spécifiques à Android (contrairement au script `free -m` utilisé de façon manuelle au chapitre 2). Cette portabilité permet de faire tourner exactement le même code de mesure sur un smartphone via Termux et sur un poste de développement Linux/Mac, ce qui facilite le débogage.

`rich` améliore la lisibilité de la sortie terminal (mise en forme, couleurs, tableaux) sans quoi une session de chat en texte brut serait difficile à suivre, en particulier lorsque plusieurs métriques doivent être affichées après chaque réponse. `prompt-toolkit` gère la saisie utilisateur de façon plus robuste que la fonction `input()` native de Python (historique de commandes, édition de ligne). Enfin, `pandas`, bien que listé à titre indicatif dans `requirements.txt`, n'est en réalité importé nulle part dans le code actuel (`chatbot.py`, `benchmark.py` et `utils.py` n'y font aucune référence) : les statistiques agrégées présentées en section 3.5.3 (moyennes, écarts-types) sont calculées directement avec le module standard `statistics`. La ligne correspondante a d'ailleurs été commentée dans `requirements.txt` au cours des tests de validation sur Termux (voir section 3.7.1), `pandas` ayant échoué à la compilation sur cet environnement sans qu'aucune fonctionnalité du prototype n'en dépende : un exemple concret de dépendance retirée une fois son inutilité constatée empiriquement, plutôt que maintenue par prudence.

3.2.3 Pipeline d'inférence

Le déroulement complet d'une requête utilisateur, depuis la saisie jusqu'à l'enregistrement des métriques, suit une séquence fixe de six étapes :

1. L'utilisateur saisit un message dans le terminal.
2. La fonction `build_chat_prompt()` (définie dans `utils.py`) assemble ce message avec l'historique de conversation et l'instruction système, selon un format compatible avec le modèle chargé (ChatML ou format Gemma).
3. Ce prompt est transmis en argument (`-p`) à un nouveau sous-processus `llama-cli`, lancé via `subprocess.Popen` avec les paramètres de génération (nombre de tokens, température, taille de contexte, nombre de threads) passés en ligne de commande ; contrairement à un appel de méthode sur un objet modèle déjà chargé, cette

approche recharge intégralement le modèle à chaque tour de parole (voir section 3.3.1).

4. Les tokens générés sont lus caractère par caractère depuis la sortie standard du sous-processus et affichés à l'écran au fur et à mesure de leur production, plutôt que d'attendre la réponse complète, ce qui améliore la latence perçue par l'utilisateur, même si la latence totale de génération reste identique.
5. Une fois le sous-processus terminé, l'objet `InferenceMetrics` est construit à partir des statistiques imprimées par `llama-cli` sur sa sortie d'erreur (temps de prefill et de decode), avec un repli par horodatage si ce format n'est pas reconnu (voir section 3.3.2).
6. Ces métriques sont censées être ajoutées de façon incrémentale au fichier `results/metrics.json` via `save_metrics()` ; en pratique, cette journalisation automatique présente un défaut identifié lors de la validation du prototype (section 3.8.3) et ne fonctionne pas de façon fiable en mode interactif : un point documenté plutôt que dissimulé, dans la mesure où il n'affecte pas la fonctionnalité de conversation elle-même, seulement sa traçabilité a posteriori.

Cette chaîne de traitement est volontairement linéaire et sans dépendance réseau à aucune étape : aucun appel externe n'intervient entre la saisie utilisateur et l'affichage de la réponse, ce qui constitue la propriété centrale que ce chapitre cherche à démontrer.

3.2.4 Format de prompt

Un point technique important, souvent sous-estimé dans les prototypes de ce type, est la construction du prompt transmis au modèle. Un LLM instruction-tuned (comme LLaMA 3.2 ou Gemma 2, utilisés dans ce PIR) attend un format de balisage précis pour distinguer les tours de parole du système, de l'utilisateur et de l'assistant ; un prompt mal formaté dégrade fortement la qualité des réponses, même si le modèle est techniquement capable de produire une bonne réponse au format attendu. Le module `utils.py` implémente donc un constructeur de prompt unique, conçu pour rester compatible avec les principaux formats d'instruction rencontrés dans ce PIR :

```
def build_chat_prompt(messages: list[dict], system_prompt: str = "") -> str:
    """
    Construit un prompt au format chat compatible avec les modèles instruction.
    Supporte les formats Gemma, LLaMA, ChatML.
    """
    prompt = ""
    if system_prompt:
        prompt += f"<|system|>\n{system_prompt}\n"

    for msg in messages:
        role = msg["role"]
```

```
content = msg["content"]
if role == "user":
    prompt += f"<|user|>\n{content}\n<|assistant|>\n"
elif role == "assistant":
    prompt += f"{content}\n"

return prompt
```

Cette fonction reçoit une liste de messages structurés (chacun associé à un rôle “user” ou “assistant”) ainsi qu’une instruction système optionnelle, et produit une chaîne de texte unique balisée avec les jetons <|system|>, <|user|> et <|assistant|>. Ce balisage explicite indique au modèle où commence et où se termine chaque tour de parole, ce qui lui permet de générer une continuation cohérente avec le rôle attendu (une réponse d’assistant, et non une nouvelle question). L’instruction système n’est insérée qu’une seule fois, en tête de prompt, car elle définit le comportement général attendu du modèle pour l’ensemble de la conversation plutôt que pour un tour de parole particulier.

3.3 Module de chatbot (chatbot.py)

Le fichier chatbot.py constitue le point d’entrée principal du prototype : il regroupe le chargement du modèle, la fonction de génération instrumentée et la définition des prompts système propres à chaque tâche. Cette section détaille ces trois éléments dans l’ordre où ils interviennent lors de l’exécution, du chargement initial du modèle jusqu’à la production d’une réponse mesurée, avant de présenter en section 3.4 les trois modes d’utilisation qui s’appuient sur ces fonctions.

3.3.1 Chargement du modèle

Contrairement à une implémentation basée sur llama-cpp-python, où le chargement du modèle constitue une opération coûteuse mais unique en début de session, chatbot.py ne charge jamais le modèle lui-même : cette responsabilité est déléguée à llama-cli, invoqué à chaque tour de parole (voir section 3.2.1). La fonction load_backend() se limite donc à une vérification de disponibilité :

```
def load_backend(model_path: str, llama_cli_path: str, n_ctx: int = 2048, n_threads: int = 4):
    """Vérifie la disponibilité du binaire llama-cli et du modèle GGUF."""
    if not os.path.exists(llama_cli_path):
        ... # message d'erreur détaillé + instructions de compilation
        sys.exit(1)

    if not os.path.exists(model_path):
        ... # message d'erreur détaillé
        sys.exit(1)
```

```
backend = {
    "llama_cli": llama_cli_path,
    "model_path": model_path,
    "n_ctx": n_ctx,
    "n_threads": n_threads,
}
return backend, model_name
```

Le code source complet de cette fonction, y compris les messages d'erreur affichés à l'utilisateur en cas d'échec, est fourni en Annexe A.

Cette fonction porte un nom volontairement différent de son équivalent `llama-cpp-python` (`load_model()`) pour refléter une différence de nature, et non seulement d'implémentation : elle ne charge aucun modèle en mémoire. Elle se contente de vérifier que le binaire `llama-cli` et le fichier GGUF existent bien aux chemins indiqués, puis retourne un simple dictionnaire de configuration (`backend`) réutilisé à chaque appel de `generate_response()` (section 3.3.2). Le modèle lui-même n'est chargé qu'au moment où un sous-processus `llama-cli` est effectivement lancé, à chaque tour de parole. Cette architecture a un coût direct en performance : le modèle est rechargé depuis le stockage à chaque échange plutôt qu'une seule fois au démarrage, comme le ferait `Llama(model_path=...)` avec `llama-cpp-python`, mais ce coût a été jugé acceptable au regard du gain de fiabilité obtenu sur Termux/Android (voir section 3.2.1) ; son impact concret sur la latence perçue est discuté en section 3.8.3.

3.3.2 Génération avec mesure de performances

La fonction de génération constitue le cœur du prototype : elle ne se contente pas d'appeler le modèle, elle instrumente précisément chaque inférence pour produire les métriques comparables à celles du chapitre 2 :

```
cmd = [
    backend["llama_cli"],
    "-m", backend["model_path"],
    "-p", prompt,
    "-n", str(max_tokens),
    "--temp", str(temperature),
    "--top-k", "40",
    "--top-p", "0.95",
    "-c", str(backend["n_ctx"]),
    "-t", str(backend["n_threads"]),
    "--no-display-prompt",
]

proc = subprocess.Popen(
    cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
```

```
    text=True, bufsize=1,  
)  
# Lecture caractère par caractère + suivi de la RAM du sous-processus  
# (psutil.Process(proc.pid)), puis extraction des métriques depuis les  
# statistiques imprimées par llama-cli sur sa sortie d'erreur.
```

Le code source complet de `generate_response()` (lecture caractère par caractère du flux de sortie, suivi de la RAM du sous-processus via `psutil`, extraction des métriques par expression régulière avec `repli` par horodatage en cas d'échec du parsing) est fourni en Annexe A.

Cette fonction ne fait plus appel à une API Python (`llm(prompt, ...)`) mais construit une liste d'arguments de ligne de commande (`cmd`) transmise à `subprocess.Popen`, exactement comme le ferait un utilisateur invoquant `llama-cli` manuellement dans un terminal, une approche qui réutilise directement le binaire déjà compilé et validé au chapitre 2. La sortie du sous-processus est lue caractère par caractère (`proc.stdout.read(1)`) plutôt qu'en une seule fois, ce qui permet de continuer à afficher la réponse progressivement à l'écran (streaming), un comportement équivalent à celui qu'offrirait nativement l'API `llama-cpp-python`. Le suivi de la RAM utilise `psutil.Process(proc.pid)` pour interroger la mémoire résidente (RSS) du sous-processus `llama-cli` à chaque itération de lecture, et en conserve le maximum observé (`peak_rss_mb`) plutôt qu'une simple mesure avant/après : cette valeur correspond à l'empreinte mémoire du processus qui héberge réellement le modèle, distincte de celle du processus Python principal (qui, lui, reste quasiment inchangée puisqu'il ne fait que piloter le sous-processus). Les métriques temporelles (`prefill`, `decode`) sont, dans la mesure du possible, extraites directement des statistiques que `llama-cli` imprime lui-même sur sa sortie d'erreur en fin d'exécution, via les expressions régulières `_PROMPT_EVAL_RE` et `_EVAL_RE` qui couvrent les deux formats de journalisation rencontrés selon la version de `llama.cpp` ; si ce format n'est pas reconnu, la fonction se rabat sur une estimation moins précise basée sur les horodatages Python et une approximation grossière du nombre de tokens à partir du nombre de mots.

Le principe de mesure, lorsqu'il aboutit, reprend la distinction `prefill/decode` établie au chapitre 2 (section 3.1), mais la source principale de ces temps n'est plus calculée par le code Python lui-même : elle est lue directement dans les statistiques que `llama-cli` imprime en fin d'exécution, cet outil effectuant la même distinction en interne. L'instant `first_token_time`, capturé côté Python dès la réception du premier caractère de sortie, ne sert alors que de repère d'affichage (déclenchement du streaming visuel) et de valeur de `repli` si les statistiques de `llama-cli` ne peuvent pas être extraites. Les paramètres `--top-k 40` et `--top-p 0.95` contrôlent la diversité de l'échantillonnage des tokens générés : `top-k` restreint le tirage aux 40 tokens les plus probables à chaque étape, et `top-p` (nucleus sampling) restreint ce tirage au sous-ensemble de tokens dont la probabilité cumulée atteint 95 %. Cette combinaison, standard dans la littérature sur la génération de texte,

évite à la fois des réponses trop répétitives (température ou top-k trop bas) et des réponses incohérentes (top-k ou top-p trop permissifs).

3.3.3 Système de prompt par tâche

Plutôt que d'utiliser un unique prompt système générique pour les trois cas d'usage, le prototype associe une instruction système spécifique à chaque tâche, ce qui améliore sensiblement la qualité et la pertinence des réponses obtenues, en particulier pour les tâches structurées (résumé, classification) où une consigne précise contraint fortement le format de sortie attendu :

```
SYSTEM_PROMPT = (  
    "Tu es un assistant IA embarqué, exécuté localement sur un smartphone "  
    "sans connexion internet. Tu réponds de manière concise et précise en français. "  
    "Limite tes réponses à 3-4 phrases maximum sauf si l'utilisateur demande plus de détails."  
)  
  
TASK_PROMPTS = {  
    "chat": "", # Utilise SYSTEM_PROMPT  
    "summary": (  
        "Tu es un assistant spécialisé dans le résumé de textes. "  
        "Résume le texte fourni en 3-5 points clés, en français, de manière concise."  
    ),  
    "classification": (  
        "Tu es un classificateur de sentiment. Analyse le texte fourni et réponds "  
        "uniquement par : [POSITIF], [NÉGATIF], ou [NEUTRE], suivi d'un score de "  
        "confiance en pourcentage et d'une justification en une phrase."  
    ),  
}
```

Le prompt système par défaut (SYSTEM_PROMPT) contraint explicitement la longueur des réponses à 3-4 phrases : cette limite n'est pas arbitraire, elle vise à réduire le temps de génération (moins de tokens à produire signifie une latence perçue plus faible) tout en restant adaptée à un usage conversationnel sur petit écran, où de longs pavés de texte nuisent à la lisibilité. Le prompt de classification pousse cette logique de contrainte plus loin en imposant un format de sortie rigide ([POSITIF], [NÉGATIF] ou [NEUTRE], suivi d'un score et d'une justification), ce qui facilite un traitement automatisé ultérieur de la réponse (par exemple son extraction par une expression régulière) sans avoir à interpréter un texte libre.

3.4 Cas d'usage implémentés

S'appuyant sur les fonctions de chatbot.py décrites en section 3.3, le prototype expose trois modes d'utilisation accessibles depuis une interface unique : un mode de conversation

libre, un mode de résumé de texte et un mode de classification de sentiment. Cette section illustre chacun de ces modes par un exemple d'exécution réel, accompagné des métriques de performance associées, afin de montrer non seulement que chaque fonctionnalité fonctionne, mais aussi comment elle se comporte en usage concret sur l'appareil testé.

3.4.1 Mode Chat interactif (Q/R libre)

Le mode chat maintient un historique de conversation complet en mémoire (la liste Python `conversation`), qui est reconstruit à chaque tour de parole via `build_chat_prompt()`. Cette conservation de l'historique complet, plutôt qu'un simple traitement d'un message isolé, est ce qui distingue un véritable chatbot conversationnel d'un simple outil de question-réponse ponctuelle : le modèle peut faire référence à ce qui a été dit précédemment dans la même session. Des commandes spéciales, préfixées par `/`, permettent de basculer vers les autres modes ou d'afficher les statistiques de la dernière réponse sans interrompre la session :

Mode CHAT interactif

Tapez votre message et appuyez sur Entrée.

Commandes : `/résumé`, `/classify`, `/stats`, `/quit`

Vous : Qu'est-ce que la quantification INT4 ?

Assistant : La quantification INT4 (4 bits par paramètre) est une technique de compression qui représente les poids d'un réseau de neurones sur 4 bits au lieu de 32 bits (FP32) ou 16 bits (FP16). Elle réduit la taille du modèle d'environ 75 % avec une perte de qualité généralement inférieure à 2 % sur les benchmarks standards. C'est le format utilisé dans ce PIR (Q4_K_M).

12,4 tok/s | 3,2s | 780 Mo (pic sous-processus)

Cet exemple illustre plusieurs propriétés observées empiriquement dans ce prototype : la réponse respecte la contrainte de longueur imposée par le prompt système (quatre phrases), et le contenu reste factuellement correct sur une question technique directement liée à ce PIR : un résultat cohérent avec les résultats de validation détaillés en section 3.8.2, où les questions factuelles simples et directement liées au domaine du PIR se sont révélées la catégorie la plus fiable du prototype. La ligne de métriques affichée diffère volontairement de celle d'une implémentation basée sur `llama-cpp-python` : la valeur de RAM (780 Mo dans cet exemple) ne représente pas une variation avant/après dans le processus Python principal, mais le pic de mémoire résidente observé dans le sous-processus `llama-cli` pendant la génération (voir section 3.3.2) : une mesure plus représentative de l'empreinte mémoire réelle du modèle, puisque celui-ci ne vit jamais dans le processus Python lui-même.

La gestion du contexte ne suit aucune logique de fenêtre glissante : contrairement à ce qu’une implémentation plus sophistiquée pourrait faire, aucun mécanisme du prototype ne retire automatiquement les messages les plus anciens de l’historique de conversation lorsque la limite de 2048 tokens fixée à l’initialisation (-c 2048) approche. La conséquence, confirmée empiriquement lors de la validation du prototype (section 3.8.3), n’est donc pas un oubli progressif des premiers échanges mais un échec net et bloquant : au-delà d’un certain nombre de tours de parole, llama-cli refuse purement et simplement de traiter la requête suivante, avec un message d’erreur explicite (request (X tokens) exceeds the available context size (2048 tokens)), ce qui interrompt immédiatement la session en cours. Cette absence de dégradation gracieuse constitue une limite réelle du prototype dans son état actuel, documentée plus en détail en section 3.8.3.

3.4.2 Mode Résumé de texte

Le mode résumé accepte le texte à traiter selon deux modalités : une saisie interactive terminée par une ligne vide, ou un fichier texte fourni via l’option --input-file. Cette double modalité répond à deux usages distincts : la saisie interactive convient à un test rapide ou à un texte court copié depuis une autre application, tandis que l’option fichier permet de traiter un document plus long sans les contraintes de saisie d’un terminal.

```
# Usage depuis le CLI
python chatbot.py --model models/llama-3.2-1b-q4_k_m.gguf --task summary
# Ou commande inline depuis le chat
Vous : /résumé L'intelligence artificielle embarquée désigne...
```

Le second exemple illustre une particularité de l’implémentation : la commande /résumé est accessible directement depuis le mode chat, sans relancer le programme avec un argument --task différent. Ce choix de conception évite à l’utilisateur d’interrompre une session en cours simplement pour obtenir un résumé ponctuel, ce qui reflète un usage réaliste où les différentes fonctionnalités d’un assistant embarqué sont généralement attendues au sein d’une seule et même interface.

3.4.3 Mode Classification de sentiment

Le mode classification illustre le cas d’usage le plus contraint des trois : contrairement au chat ou au résumé, où la réponse attendue est un texte libre de longueur variable, la classification impose une sortie structurée et volontairement brève, ce qui se traduit par un paramètre max_tokens réduit à 100 dans le code (contre 512 par défaut pour le chat) et par la désactivation du streaming (stream=False), puisqu’une réponse aussi courte n’apporte pas de bénéfice perceptible à un affichage progressif token par token.

```
python chatbot.py --model models/llama-3.2-1b-q4_k_m.gguf --task classify
Mode CLASSIFICATION de sentiment
Texte : Ce smartphone est excellent, la batterie tient toute la journée.
```

Résultat : [POSITIF] – Confiance : 94 % – Le texte exprime une satisfaction claire sur deux aspects spécifiques du produit (performance et autonomie).

8.7 tok/s | 2.1s

Le débit de decode plus faible observé ici (8,7 tok/s) par rapport au mode chat (12,4 tok/s dans l'exemple précédent) n'indique pas une dégradation de performance du modèle : il s'explique par le fait qu'une réponse de classification, plus courte, atteint une proportion plus importante de tokens de ponctuation et de structure (crochets, pourcentage) dont la génération peut légèrement varier en vitesse par rapport à du texte narratif continu, un effet de mesure à relativiser sur un échantillon aussi court.

3.5 Module de benchmark (benchmark.py)

Distinct du module de chat, `benchmark.py` a pour rôle de produire des mesures de performance reproductibles, indépendamment du contenu saisi par un utilisateur. Cette section présente successivement le jeu de prompts standardisés utilisé, le protocole de mesure appliqué à chaque run (y compris la détection du throttling thermique), la présentation des résultats à l'écran, puis leur sauvegarde persistante, soit l'ensemble de la chaîne allant de la définition du test à l'archivage de son résultat.

3.5.1 Prompts de test standardisés

Contrairement au mode chat, où le contenu des échanges dépend de l'utilisateur et n'est donc pas reproductible d'une session à l'autre, le module de benchmark repose sur un jeu de prompts fixes, conçus pour solliciter le modèle à des niveaux de charge croissants et comparables entre appareils :

Type	Prompt	Tokens attendus	Description
short	"Quelle est la capitale de la France ?"	30	Question courte
medium	"Explique en 5 points les avantages de l'IA embarquée."	200	Charge modérée
long	"Rédige un tutoriel llama.cpp sur Android via Termux..."	500	Forte charge
reasoning	Problème de train Paris-Lyon (512 km, calcul croisement)	300	Test de raisonnement

Tableau 3.2 : Jeu de prompts standardisés utilisés par le module de benchmark

Ces quatre catégories ont été choisies pour couvrir des profils de génération distincts : le prompt short teste la latence de première réponse sur une question factuelle triviale, représentative d'un usage ponctuel de type assistant vocal ; le prompt medium sollicite

une génération structurée de longueur intermédiaire, proche de ce qui est attendu du mode résumé ; le prompt long pousse la génération vers sa limite pratique, ce qui permet d'observer d'éventuels effets de dégradation thermique sur une durée d'inférence plus longue, comme mesuré au chapitre 2 (section 3.3) ; enfin, le prompt reasoning, construit autour d'un problème arithmétique de croisement de trains, ne teste pas la vitesse mais la capacité du modèle à produire un raisonnement correct en plusieurs étapes, une limite déjà identifiée pour les modèles de moins de 2 milliards de paramètres au chapitre 1 (section 1.3).

3.5.2 Protocole de benchmark

Le protocole de mesure implémenté dans `benchmark.py` reprend la même logique de séparation prefill/decode que le module de chat, mais y ajoute un mécanisme de détection automatique du throttling thermique, phénomène central du chapitre 2 (section 3.10.1) :

```
for chunk in llm(
    prompt,
    max_tokens=max_tokens,
    temperature=0.1, # Basse température pour reproductibilité
    stream=True,
    stop=["</s>", "<|user|>"],
):
    if first_token_time is None:
        first_token_time = time.time()
    token_count += 1
    print(".", end="", flush=True)

# Détection du throttling
decode_tps = token_count / max(decode_time, 0.01)
throttling = False
if prev_decode_tps and decode_tps < prev_decode_tps * 0.85:
    throttling = True # Dégradation > 15 % = throttling probable
```

Le code source complet de `run_single_benchmark()` est fourni en Annexe B.

Le mécanisme de détection du throttling repose sur une comparaison relative plutôt qu'absolue : chaque run de benchmark est comparé au débit de decode du run précédent (`prev_decode_tps`), et une dégradation supérieure à 15 % déclenche un signalement. Ce seuil relatif, plutôt qu'un seuil de débit absolu en tokens/s, a l'avantage de fonctionner indépendamment de la puissance de l'appareil testé : un flagship rapide et un appareil d'entrée de gamme plus lent peuvent tous deux être correctement diagnostiqués, puisque c'est la variation par rapport à leur propre performance de référence qui est observée, non leur débit brut. Ce principe reprend l'idée du chapitre 2 (variation relative du débit de decode, avec des dégradations de -17 % à -19 % mesurées sur le Galaxy S26 et le Galaxy A16 respectivement), mais s'en distingue par sa référence : le chapitre 2 compare

une baseline à l'état après 5 minutes de charge soutenue (`throttling_rigoureux.sh`), tandis que `benchmark.py` compare chaque run au précédent. C'est donc un indicateur de signalement, et non une mesure de throttling au sens du chapitre 2. La température d'échantillonnage est volontairement abaissée à 0,1 (contre 0,7 pour le mode chat) afin de maximiser la reproductibilité des réponses générées d'un run à l'autre : à une température aussi basse, le modèle privilégie systématiquement les tokens les plus probables, ce qui limite la variabilité du nombre de tokens générés et donc du temps de mesure, un facteur de bruit expérimental que l'on cherche à minimiser dans un protocole de benchmark. Contrairement au mode chat (section 3.3.2), dont l'invocation de `llama-cli` en sous-processus ne définit aucune séquence d'arrêt explicite, l'appel `llm(...)` de `benchmark.py`, qui utilise directement les bindings `llama-cpp-python`, plutôt que `llama-cli`, car ce module n'a pas besoin d'interface interactive et bénéficie donc pleinement de l'API Python (voir section 3.2.2), définit une liste `stop=["</s>", "<|user|>"]` : si le modèle génère un jeton correspondant au début d'un nouveau tour de parole ou à un jeton de fin de séquence, la génération s'interrompt immédiatement, ce qui borne la durée de chaque run et évite qu'une dérive du modèle ne fausse la mesure de débit.

3.5.3 Affichage des résultats

Les résultats de chaque session de benchmark sont présentés sous forme de tableau récapitulatif directement dans le terminal, avec un calcul de moyenne et d'écart-type par type de prompt, similaire dans sa forme aux tableaux de résultats du chapitre 2 :

RÉSULTATS DU BENCHMARK							
Type	Run	Prefill	Decode	Tokens	Temps	RAM	Throttle
short	1	46.2/s	12.8/s	28	2.3s	2245Mo	Non
short	2	45.9/s	12.4/s	28	2.4s	2247Mo	Non
medium	1	47.1/s	12.6/s	187	15.2s	2251Mo	Non
long	1	46.8/s	11.2/s	463	42.1s	2254Mo	Oui
short	$\mu=12.6$ tok/s $\sigma=0.28$ min=12.4 max=12.8						

La colonne "Throttle" illustre directement le mécanisme de détection décrit en section 3.5.2 : le run associé au prompt long, seul à dépasser 40 secondes d'inférence continue, est le seul signalé comme potentiellement affecté par une dégradation thermique, ce qui est cohérent avec l'intuition que le throttling apparaît après une charge soutenue plutôt qu'une inférence brève.

3.5.4 Sauvegarde des résultats

Chaque session de benchmark est enregistrée dans un fichier JSON horodaté, accompagné de métadonnées système capturées au moment du test :

```
def save_benchmark_results(results, model_name):
    """Sauvegarde les résultats en JSON avec métadonnées système."""
    data = {
        "model": model_name,
        "timestamp": time.strftime("%Y-%m-%dT%H:%M:%S"),
        "system": get_system_ram_mb(), # RAM totale + disponible
        "results": [asdict(r) for r in results],
    }
    with open(f"results/benchmark_{model_name}_{timestamp}.json", "w") as f:
        json.dump(data, f, indent=2, ensure_ascii=False)
```

Le choix d'un format JSON plutôt qu'une base de données ou un simple fichier texte répond à un besoin de portabilité et de traçabilité : chaque fichier de résultat est autosuffisant (il inclut le nom du modèle testé, l'horodatage et l'état de la RAM système au moment du test), ce qui permet de comparer a posteriori des sessions de benchmark exécutées à des dates différentes ou sur des appareils différents, sans risque de confusion sur les conditions de mesure. L'inclusion explicite de la RAM système totale et disponible est particulièrement importante sur Android, où la quantité de RAM réellement disponible pour une application varie selon les autres processus actifs au moment du test, un facteur de variance déjà identifié au chapitre 2 pour expliquer certaines valeurs de throttling aberrantes attribuées au gouverneur de fréquence Android plutôt qu'à une réelle contrainte thermique.

3.6 Métriques collectées (utils.py)

Les modules `chatbot.py` et `benchmark.py`, présentés dans les sections précédentes, s'appuient tous deux sur une structure de données commune pour représenter les métriques de performance d'une inférence. Cette section décrit cette structure, définie dans `utils.py`, ainsi que la méthode de formatage textuel qui permet de l'afficher de façon lisible directement dans le terminal.

3.6.1 Structure InferenceMetrics

L'ensemble des mesures de performance produites par le prototype, qu'il s'agisse d'un tour de chat ou d'un run de benchmark, est encapsulé dans une unique structure de données :

```
@dataclass
class InferenceMetrics:
    model_name: str
    prompt_tokens: int      # Tokens dans le prompt
    generated_tokens: int   # Tokens générés
```

```

load_time_s: float      # Temps de rechargement du modèle (estimé)
prefill_time_s: float   # Temps de traitement du prompt
decode_time_s: float    # Temps de génération des tokens
total_time_s: float     # Temps total
prefill_speed_tps: float # tokens/s pendant le prefill
decode_speed_tps: float  # tokens/s pendant le decode
ram_before_mb: float    # RAM avant inférence
ram_after_mb: float     # RAM après inférence
ram_delta_mb: float     # Variation de RAM
cpu_percent: float      # Usage CPU moyen

```

L'utilisation d'une dataclass Python plutôt qu'un simple dictionnaire présente un double avantage pour ce prototype : elle garantit, par typage statique, que chaque métrique attendue est bien présente et du bon type à chaque appel, ce qui évite des erreurs silencieuses (une métrique oubliée ou mal nommée) qui seraient difficiles à détecter dans un dictionnaire non structuré ; elle permet également une sérialisation directe en JSON via la fonction `asdict()`, utilisée par le module de benchmark pour l'enregistrement des résultats. La structure distingue systématiquement les métriques de RAM avant et après inférence (`ram_before_mb`, `ram_after_mb`) plutôt que la seule variation, ce qui permet de vérifier, sur une session longue, si la consommation mémoire de base dérive progressivement d'un tour de parole à l'autre : un signe éventuel de fuite mémoire qui ne serait pas visible si seul le delta était conservé.

3.6.2 Résumé textuel automatique

Pour faciliter la lecture immédiate des métriques après chaque réponse, sans devoir consulter le fichier JSON, la structure `InferenceMetrics` expose une méthode de formatage textuel :

```

def summary(self) -> str:
    return (
        f"\n{'-'*50}\n"
        f"  Modèle      : {self.model_name}\n"
        f"  Prompt      : {self.prompt_tokens} tokens\n"
        f"  Généré      : {self.generated_tokens} tokens\n"
        f"  Chargement   : {self.load_time_s:.2f}s (estimé)\n"
        f"  Prefill      : {self.prefill_time_s:.2f}s "
        f"({self.prefill_speed_tps:.1f} tok/s)\n"
        f"  Decode       : {self.decode_time_s:.2f}s "
        f"({self.decode_speed_tps:.1f} tok/s)\n"
        f"  Total        : {self.total_time_s:.2f}s\n"
        f"  RAM delta    : +{self.ram_delta_mb:.0f} Mo "
        f"({self.ram_after_mb:.0f} Mo total)\n"
        f"  CPU usage    : {self.cpu_percent:.1f}%\n"
        f"{'-'*50}"
    )

```

)

Le code source complet de `utils.py`, y compris les fonctions de mesure système (`get_ram_usage_mb`, `get_system_ram_mb`, `safe_cpu_percent`) et de sauvegarde (`save_metrics`), est fourni en Annexe C.

Cette méthode est appelée directement depuis les modes résumé et classification (section 3.4) pour afficher un rapport de performance complet après chaque génération, alors que le mode chat n’affiche qu’une version condensée sur une seule ligne (débit, temps total, delta RAM) afin de ne pas alourdir visuellement une conversation censée rester fluide. Ce choix illustre un arbitrage volontaire entre exhaustivité de l’information et lisibilité de l’interface, tranché différemment selon le cas d’usage.

3.7 Installation et utilisation

Après avoir détaillé l’architecture et le fonctionnement interne du prototype, cette section décrit la procédure d’installation et les différentes façons de le lancer, depuis les prérequis logiciels jusqu’aux paramètres avancés disponibles en ligne de commande. Ces étapes complètent, du point de vue de l’utilisateur final, les choix d’implémentation justifiés dans les sections précédentes.

Le code source complet du prototype (`chatbot.py`, `benchmark.py`, `utils.py`) est publié sur le dépôt public du projet, <https://github.com/on-device-llm/llm-smartphone>, qui constitue la référence à jour et exécutable ; les Annexes A à C en détaillent le contenu plutôt que d’en reproduire l’intégralité.

3.7.1 Prérequis

L’installation du prototype suit directement la procédure Termux déjà détaillée au chapitre 2 (section 1.1), à laquelle s’ajoutent les dépendances Python propres à ce prototype :

```
# Sur Android (Termux)
pkg install python
pip install -r requirements.txt
# requirements.txt
# llama-cpp-python>=0.2.90 (utilisé uniquement par benchmark.py)
# psutil>=5.9.0
# rich>=13.7.0
# prompt_toolkit>=3.0.43
# pandas>=2.0.0 (commenté, non importé, retiré après échec de compilation sous Termux)
```

Le fichier `requirements.txt` complet est fourni en Annexe D. La contrainte de version minimale `llama-cpp-python>=0.2.90` n’est pas arbitraire : elle correspond à la première version des bindings intégrant de façon stable le support du streaming token par token utilisé dans `benchmark.py`, une fonctionnalité indisponible dans les versions antérieures de

la bibliothèque. Fixer cette borne dans `requirements.txt` évite qu’une installation avec une version plus ancienne échoue silencieusement ou produise un comportement dégradé.

L’installation sur les appareils Android testés (Termux) n’a pas été immédiate : plusieurs dépendances transitives ont nécessité une intervention manuelle avant de fonctionner correctement, comme documenté dans le journal de validation du prototype. `numpy` (dépendance de `llama-cpp-python`, donc de `benchmark.py`) échoue à la compilation sous Termux, la bibliothèque C d’Android (Bionic) ne déclarant pas certaines fonctions complexes `long double (ccosl, csinl)` attendues par le code source de `numpy` ; la solution retenue a été d’installer la version précompilée fournie par le gestionnaire de paquets Termux (`pkg install python-numpy`) plutôt que de laisser `pip` compiler depuis les sources. Une contrainte similaire est apparue avec `psutil` sur l’un des appareils testés, résolue de la même façon (`pkg install python-psutil`). Ces incidents, mineurs mais réels, illustrent une difficulté pratique peu documentée dans la littérature sur l’inférence embarquée : la compatibilité d’un environnement Python complet sur Android dépend autant de la disponibilité de paquets précompilés que de la faisabilité théorique de l’inférence elle-même.

3.7.2 Lancement

Le prototype expose un unique point d’entrée (`chatbot.py`), dont le comportement varie selon les arguments de ligne de commande fournis, ce qui évite la prolifération de scripts distincts pour chaque cas d’usage :

```
# Mode démo sans modèle (réponses simulées)
python chatbot.py --mock

# Chat interactif avec modèle réel
python chatbot.py --model ~/models/llama-3.2-1b-instruct-q4_k_m.gguf

# Mode résumé
python chatbot.py --model ~/models/llama-3.2-1b-instruct-q4_k_m.gguf --task summary

# Mode classification
python chatbot.py --model ~/models/llama-3.2-1b-instruct-q4_k_m.gguf --task classify

# Benchmark complet
python benchmark.py --model ~/models/llama-3.2-1b-instruct-q4_k_m.gguf --runs 3

# Benchmark en mode démo
python benchmark.py --mock --runs 3
```

Le mode `--mock`, disponible aussi bien pour le chatbot que pour le benchmark, mérite une attention particulière : il permet de tester l’intégralité de l’interface (saisie, affichage, enchaînement des commandes) sans charger de modèle réel, en simulant des réponses à une vitesse de génération réaliste (10 à 15 tokens/s). Ce mode s’est révélé utile à deux étapes du développement de ce PIR : lors du développement initial de l’interface, avant même d’avoir un modèle GGUF téléchargé et validé, et lors de démonstrations ou de vérifications rapides ne nécessitant pas de solliciter réellement le processeur de l’appareil de test.

3.7.3 Paramètres avancés

Au-delà du choix du modèle et de la tâche, plusieurs paramètres en ligne de commande permettent d'ajuster le comportement du prototype sans modifier le code source. Les deux points d'entrée (`chatbot.py` et `benchmark.py`) exposent chacun leur propre jeu de paramètres, résumés séparément ci-dessous :

Paramètre (<code>chatbot.py</code>)	Défaut	Description
<code>--model</code>	-	Chemin vers le fichier GGUF
<code>--llama-cli</code>	<code>~/llama.cpp/build/bin/llama-cli</code>	Chemin vers le binaire llama-cli
<code>--task</code>	chat	Tâche à exécuter (chat, summary, classify)
<code>--n-ctx</code>	2048	Taille du contexte en tokens
<code>--threads</code>	4	Nombre de threads CPU (recommandé : nproc)
<code>--max-tokens</code>	512	Tokens maximum générés par réponse
<code>--input-file</code>	-	Fichier texte d'entrée pour le mode résumé
<code>--no-stream</code>	False	Désactiver l'affichage token par token
<code>--mock</code>	False	Mode démo sans modèle réel

Tableau 3.3 : Paramètres en ligne de commande de `chatbot.py`

Paramètre (<code>benchmark.py</code>)	Défaut	Description
<code>--model</code>	-	Chemin vers le fichier GGUF
<code>--runs</code>	3	Répétitions par type de prompt
<code>--prompts</code>	all	Types de prompts testés (short, medium, long, reasoning)
<code>--threads</code>	4	Nombre de threads CPU
<code>--n-ctx</code>	2048	Taille du contexte en tokens
<code>--mock</code>	False	Mode démo sans modèle réel

Tableau 3.4 : Paramètres en ligne de commande de `benchmark.py`

Le paramètre `--threads`, dont la valeur recommandée est `nproc` (nombre de cœurs disponibles sur l'appareil), reprend directement la logique de parallélisation déjà appliquée à la compilation de `llama.cpp` au chapitre 2 : allouer un thread par cœur physique maximise l'utilisation du CPU pendant l'inférence, au prix d'une consommation énergétique plus élevée, un compromis discuté en détail au chapitre 2 (section 3.8).

3.8 Résultats observés sur le prototype

Les sections précédentes ont détaillé l'implémentation du prototype ; celle-ci en présente les résultats observés lors de son exécution effective sur le corpus d'appareils de ce PIR. Trois angles sont couverts successivement : les performances brutes de génération en mode chat interactif, la qualité perçue des réponses selon le cas d'usage, et les limites

pratiques rencontrées lors d'un usage prolongé, trois dimensions complémentaires à la seule mesure de débit déjà présentée au chapitre 2.

3.8.1 Performances sur les appareils validés (LLaMA 3.2 1B Q4_K_M, chat interactif)

Contrairement au protocole de benchmark automatisé du chapitre 2, qui a pu être exécuté sur l'ensemble des six appareils du corpus grâce à sa nature non interactive, la validation du prototype `chatbot.py` (qui nécessite une observation qualitative de chaque réponse, tour par tour) a désormais été menée à son terme sur l'ensemble des six appareils du corpus : le Galaxy S26 Ultra (Snapdragon 8 Elite, haut de gamme), le Galaxy A16 (Exynos 1330, entrée de gamme), l'Infinix Hot 60i 5G (Dimensity 6400, milieu de gamme), le Galaxy A26 (Exynos 1280, milieu de gamme), le Galaxy A71 (Snapdragon 730, milieu de gamme) et le Galaxy A73 (Snapdragon 778G, milieu de gamme), couvrant ainsi trois vendeurs de SoC distincts (Qualcomm, Samsung, MediaTek) ; le Galaxy A16 et le Galaxy A26 partagent le même vendeur (Samsung Exynos) mais sur deux puces différentes, de même que le Galaxy S26 Ultra, le Galaxy A71 et le Galaxy A73 partagent le vendeur Qualcomm sur trois segments de gamme et générations de puce différents. Le protocole complet (douze questions factuelles et de raisonnement, un exercice de résumé, six textes de classification, rejoués sur plusieurs runs indépendants) est détaillé en Annexe E ; seuls les résultats agrégés sont présentés ici.

Appareil	SoC	Prefill observé	Decode observé
Galaxy S26 Ultra	Snapdragon 8 Elite	~150 à 235 tok/s	~50 à 60 tok/s
Galaxy A16	Exynos 1330	~12,1 à 52,0 tok/s	~3,9 à 7,8 tok/s
Infinix Hot 60i 5G	Dimensity 6400	~44 à 56 tok/s	~0,8 à 13,8 tok/s
Galaxy A26	Exynos 1280	~56 à 99 tok/s	~4,6 à 16,7 tok/s
Galaxy A71	Snapdragon 730	~35 à 43 tok/s	~6 à 9 tok/s
Galaxy A73	Snapdragon 778G	~49 à 64 tok/s	~7 à 13 tok/s

Tableau 3.5 : Performances observées sur le prototype interactif, par appareil (LLaMA 3.2 1B Q4_K_M)

Ces plages de valeurs, plus larges que les moyennes ponctuelles rapportées au chapitre 2, reflètent la variabilité observée sur plusieurs tours de parole réels plutôt qu'une unique mesure de référence obtenue via `llama-bench`. Elles confirment l'écart de segment de gamme déjà établi au chapitre 2 entre les appareils, mais à un niveau de performance sensiblement inférieur à celui mesuré par `llama-bench` : ce n'est pas une contradiction avec les résultats du chapitre 2, mais une conséquence directe de l'architecture retenue pour `chatbot.py` (section 3.2.1), où le modèle est rechargé depuis le stockage à chaque tour de parole via un nouveau sous-processus `llama-cli`, plutôt que chargé une seule fois en mémoire comme le fait `llama-bench` ou le module `benchmark.py` du présent prototype

(qui utilise, lui, les bindings `llama-cpp-python` avec un modèle chargé une seule fois, voir section 3.5). Ce coût de rechargement répété, assumé comme limite du prototype (section 3.8.3), n'empêche pas le Galaxy A16, l'Infinix, l'A26, l'A71 ni l'A73 de rester utilisables en conversationnel malgré une latence par réponse nettement plus élevée que le S26 Ultra. Les bornes basses du decode sur l'Infinix (0,8 tok/s) et sur l'A26 (4,6 tok/s) sont des points isolés plutôt que représentatifs : elles correspondent à des tours où le contexte accumulé était déjà important (voir section 3.8.3 et `journal_validation_prototype.md`), et non à une limite structurelle du SoC ; l'A26 obtient d'ailleurs, sur le reste de la session, des débits légèrement supérieurs à l'A16 malgré un segment de gamme comparable, cohérent avec les mesures chapitre 2 (tableau 2.8). Le Galaxy A71, malgré un SoC déjà plus ancien (Snapdragon 730), obtient en decode chat interactif un débit (~ 6 à 9 tok/s) nettement inférieur à celui mesuré au chapitre 2 pour ce même appareil via `llama-bench` ($\sim 11,4$ à $11,7$ tok/s) : un écart cohérent avec le surcoût de rechargement par tour déjà établi pour les autres appareils, plutôt qu'une anomalie propre à cet appareil. Le Galaxy A73 suit le même schéma : son decode chat interactif (~ 7 à 13 tok/s, avec une baisse progressive au fil de la session, de ~ 13 tok/s au premier tour à ~ 7 tok/s sur les questions de raisonnement les plus tardives) reste inférieur à la mesure `llama-bench` du chapitre 2 pour ce même appareil ($13,36 \pm 0,80$ tok/s en Termux natif), cohérent avec le surcoût de rechargement par tour et avec l'allongement progressif du contexte au fil de la session.

Un test complémentaire a été réalisé sur le Galaxy S26 Ultra afin de disposer d'une latence de bout en bout comparable à celle mesurée avec LiteRT (chapitre 2, section 3.4) : la même question (« Explique-moi le concept d'intelligence artificielle en 3 phrases. ») a été posée trois fois, chaque fois dans une session neuve (contexte vide), avec le build `llama-cli b10154-0e4a03622`. Les temps totaux sont de 3,1 s, 3,3 s et 2,7 s (moyenne 3,1 s), dont environ 1,0 s de rechargement du modèle (`load_time_s`), pour un decode de 55,3 à 55,4 tok/s et un prefill de 216 à 218 tok/s. Hors rechargement, la latence est donc d'environ 2,0 s. Ces valeurs sont cohérentes avec la plage du tableau 3.5 pour cet appareil (~ 50 à 60 tok/s en decode).

3.8.2 Qualité des réponses

Contrairement à une estimation qualitative globale, la qualité des réponses a été évaluée selon le protocole détaillé en Annexe E, joué sur deux runs indépendants sur le Galaxy S26 Ultra, un run sur le Galaxy A16, un run sur l'Infinix Hot 60i 5G, un run sur le Galaxy A26, un run sur le Galaxy A71 et un run sur le Galaxy A73, soit sept échantillons indépendants au total pour la plupart des catégories testées, couvrant l'intégralité du corpus de six appareils, toujours trois vendeurs de SoC distincts (Qualcomm, Samsung, MediaTek) : le Galaxy A26 partage le vendeur Samsung Exynos avec l'A16, et le Galaxy A71 et le Galaxy A73 partagent le vendeur Qualcomm avec le S26 Ultra, sur trois segments de gamme et générations de puce différents.

Sur les sept questions factuelles simples, une seule question reste parfaitement stable sur les sept échantillons : la capitale de la France. L'année de la Révolution française, longtemps présentée comme également stable, échoue finalement sur le septième échantillon (A73, aucune réponse fournie), ce qui confirme qu'aucune question du protocole, même la plus simple en apparence, n'est totalement à l'abri d'une réponse ponctuellement incorrecte, pas même les deux questions les plus élémentaires du protocole. Une question échoue systématiquement, quel que soit l'appareil ou le run, sur l'ensemble des sept échantillons : le nombre de jours d'une année bissextile (la valeur 366 n'a jamais été produite, le modèle générant à la place des approximations telles que « 365,5 », « 365,24 », « 365,2425 », un refus pur et simple de trancher sur l'A26, une hallucination d'un tour de conversation fictif sur l'A71, ou une contamination croisée avec une autre question du protocole sur l'A73). L'auteur du *Petit Prince*, stable sur les quatre premiers échantillons, échoue sur le cinquième (A26), redevient correct sur le sixième (A71), puis échoue de nouveau sur le septième (A73, absence totale de réponse). Les questions restantes (symbole chimique de l'eau, plus grande planète, nombre de continents) illustrent la même instabilité résiduelle sur les derniers échantillons, sans motif unique : le symbole chimique de l'eau, en échec trois fois de suite (Infinix, A26, A71), redevient correct sur l'A73 ; la plus grande planète, en échec sur l'Infinix et l'A26, redevient correcte sur l'A71 avant d'échouer de nouveau sur l'A73 ; le nombre de continents, qui échouait systématiquement sur les trois premiers runs, est correctement répondu sur l'Infinix et l'A26, échoue de nouveau sur l'A71, puis n'obtient qu'une réponse partielle sur l'A73 (bon décompte, composition erronée de la liste). Cette absence de motif stable d'un échantillon à l'autre, sur ces questions précisément, reste cohérente avec l'hypothèse d'une perturbation logicielle commune aux quatre derniers appareils (voir section 3.8.3) plutôt qu'avec une difficulté intrinsèque propre à chaque question. Les scores globaux sur l'ensemble du corpus sont : S26 Ultra et A16 5/7, A71 4/7, Infinix 4/7, A26 3/7, A73 2/7. Cette plage, désormais élargie par ce septième échantillon (2 à 5 sur 7), reste malgré la variabilité question par question largement supérieure à 0. L'A73 introduit par ailleurs deux modes d'échec inédits sur l'ensemble du corpus : l'esquive pure (une formule de politesse générique sans aucune tentative de réponse, sur deux questions distinctes) et la contamination croisée totale (la réponse à une question recopie intégralement celle d'une question précédente sans rapport). Ce résultat contraste en tout cas nettement avec le raisonnement multi-étapes : sur les sept échantillons indépendants (trente-quatre tentatives sur trente-cinq questions prévues, la douzième n'ayant pu être posée sur l'Infinix en raison d'une saturation du contexte), le modèle ne produit que cinq réponses partielles au total, les vingt-neuf autres tentatives étant incorrectes, et de façon instable : les mêmes questions échouent différemment d'un run à l'autre (erreurs de calcul différentes, confusions conceptuelles différentes, jusqu'à une question posée deux fois par erreur sur l'A26 et obtenant deux réponses différentes, l'une incorrecte et l'autre partiellement correcte). Le modèle n'est donc pas simplement faible en raisonnement

multi-étapes : il est imprévisible dans sa manière de l'être. Ce résultat, désormais confirmé sur l'ensemble des six appareils du corpus et trois vendeurs de SoC indépendants, sans une seule exception, est cohérent avec les limites déjà documentées pour les modèles de moins de 2 milliards de paramètres au chapitre 1 (section 1.3).

En mode résumé, testé sur huit runs au total (trois sur le S26 Ultra, un sur l'A16, un sur l'Infinix, un sur l'A26, un sur l'A71 (ce dernier retenu après un run de contrôle en session fraîche ; un premier essai réalisé après les douze questions de la Partie A avait produit un résultat nettement plus faible, vraisemblablement du fait de l'historique déjà accumulé combiné à l'incident de build détaillé en 3.8.3), et un sur l'A73) avec le même texte source portant sur les trois piliers de l'inférence LLM embarquée, le verdict global est « Bon » (au moins trois des quatre points clés attendus présents, ou deux points complets et deux partiels) dans cinq runs sur huit, et « Partiel » dans les trois runs restants (S26 Ultra run 3, A26 et A73, ces deux derniers suivant exactement le même profil : un point complet, deux partiels, un absent). Le point le plus instable est la mention des optimisations matérielles (ARM NEON / NPU) : complète dans deux runs, absente dans un run, partiellement correcte (ARM NEON cité, NPU omis ou attribution erronée ; ou, sur l'A73, les frameworks nommés sans jamais citer ARM NEON ni NPU) dans les cinq runs restants (A16, Infinix, A26, A71, A73). Le terme « memory-mapping » ou « mmap » apparaît explicitement dans quatre des huit runs (S26 Ultra run 2, Infinix, A26, A71) ; dans les quatre autres, le principe est décrit correctement mais avec d'autres mots (dont l'A73). Les textes plus longs que le texte de test doivent, par ailleurs, être découpés manuellement par l'utilisateur avant traitement, une limite directement dictée par la fenêtre de contexte de 2048 tokens, qui n'a pas été automatisée dans ce prototype.

En mode classification, les résultats diffèrent nettement selon l'appareil : sur le Galaxy S26 Ultra, les deux runs obtiennent un score parfait de 6/6, soit 12/12 au total, y compris sur le texte volontairement piégeux (ironie : « ce qu'on appelle un service "rapide"... trois semaines d'attente »), correctement classé NÉGATIF dans les deux runs avec une confiance de 65 à 72 % ; au second run, la justification générée mentionne même explicitement un « ton sarcastique ». Sur le Galaxy A16, le score chute à 4/6, avec deux échecs qui portent précisément sur les deux cas les plus subtils du protocole : le texte ironique (classé à tort NEUTRE, avec une confiance de seulement 2 % et une justification interne contradictoire) et un texte neutre/mitigé (classé à tort NÉGATIF). Sur l'Infinix, le score s'effondre à 1/6, avec un biais systématique vers le label NÉGATIF (5 des 6 textes), y compris sur deux textes clairement positifs classés NÉGATIF à 80 % et 8 % de confiance, malgré des justifications internes qui décrivent pourtant un contenu positif, ce qui constitue une contradiction directe entre le raisonnement affiché et le label produit, plus marquée que les incohérences déjà observées sur l'A16. Sur le Galaxy A26, le score remonte à 3/6, mais selon un mode de dégradation encore différent : ni biais directionnel généralisé (comme l'Infinix) ni simple confusion de label (comme l'A16), mais

une contradiction directe et répétée entre une justification textuelle correcte et un label final erroné sur deux des trois échecs (un texte neutre justifié comme « neutre » mais étiqueté NÉGATIF, un texte positif justifié comme « très positif » mais étiqueté NEUTRE). Sur le Galaxy A71, le score retombe à 2/6, avec cette fois un biais directionnel total et sans exception : les six textes sur six sont étiquetés NÉGATIF, indépendamment de leur contenu réel, et quatre des six justifications textuelles contredisent ouvertement le label produit. Ce profil combine ainsi le biais directionnel de l’Infinix et les contradictions internes de l’A26 en un seul mode de dégradation. Sur le Galaxy A73, le score reste à 2/6, avec exactement le même profil que l’A71 : un biais directionnel total (six textes sur six étiquetés NÉGATIF) combiné à des contradictions internes sur trois des six justifications. Sur l’ensemble des sept échantillons indépendants du cas piège de l’ironie, le résultat est de six corrects sur sept (S26 Ultra ×2, Infinix, A26, A71, A73) et un seul échec (A16) ; ce chiffre doit toutefois être relativisé pour l’Infinix, l’A71 et l’A73, où le biais généralisé vers NÉGATIF rend probable un succès mécanique plutôt qu’une réelle détection de l’ironie : ces trois appareils auraient vraisemblablement classé n’importe quel texte de la même façon ; le succès de l’A26 sur ce même texte, en revanche, survient dans un run qui ne présente pas ce biais généralisé, ce qui en fait un résultat plus probant. La détection de l’ironie ne doit donc pas être présentée comme une capacité acquise du prototype, mais comme un résultat mitigé et dépendant de l’échantillon testé : plus nuancé que l’hypothèse initiale de ce PIR (l’ironie est mal détectée), mais moins optimiste qu’une conclusion tirée du seul run favorable du S26 Ultra. La dégradation du score global sur l’Infinix, l’A26, l’A71 et l’A73 est discutée plus loin comme probablement liée à un facteur logiciel commun plutôt que matériel (section 3.8.3).

Afin de vérifier que cette limite de raisonnement multi-étapes n’est pas un artefact spécifique au framework `llama.cpp` ou à l’architecture par sous-processus retenue pour ce prototype, les mêmes douze questions ont été rejouées sur le Galaxy S26 Ultra via une seconde solution d’inférence on-device, indépendante : l’application Google AI Edge Gallery. Le modèle Gemma 2 2B, initialement retenu comme référence de comparaison Google dans ce mémoire, n’étant plus proposé au téléchargement dans le catalogue de cette application au moment du test, il a été remplacé par Gemma3-1B-IT, le modèle disponible le plus proche en nombre de paramètres, afin de préserver une comparaison à échelle comparable (détail complet dans `journal_validation_prototype.md`). Le résultat confirme l’observation faite avec LLaMA 3.2 1B : sur les cinq questions de raisonnement multi-étapes, Gemma3-1B-IT échoue intégralement (0/5), tandis que sur les sept questions factuelles simples il obtient un score comparable (5/7, avec des erreurs différentes en détail mais portant sur les deux mêmes catégories de question). Cette confirmation croisée, obtenue avec un second modèle et un second framework d’inférence totalement indépendants du prototype développé dans ce PIR, renforce la conclusion que la limite de raisonnement multi-étapes observée est une caractéristique des modèles de cette échelle

de paramètres (~1 milliard), et non un défaut d'implémentation propre à `chatbot.py` ou à `llama.cpp`.

Cette confirmation croisée a ensuite été étendue aux deux autres volets du protocole de qualité, le résumé et la classification, en réutilisant exactement le même texte source et les six mêmes textes qu'en 3.8.2, sans nouveau protocole. En mode résumé, Gemma3-1B-IT ne restitue correctement qu'un seul des quatre points clés attendus (la gestion mémoire par memory-mapping) ; les trois autres restent partiels : la réduction de taille par quantification est décrite avec un facteur de bits inventé (« 768 bits à 4 bits », une valeur sans rapport avec le texte source ni avec un standard de précision courant), l'optimisation matérielle omet le NPU et attribue à tort l'usage d'ARM NEON à la fois à `llama.cpp` et à ML Kit GenAI, et la conclusion reste générique, sans reprendre les chiffres précis du texte source (modèles de 1 à 3 milliards de paramètres, déploiement depuis 2020). Ce résultat est en retrait par rapport aux runs `llama.cpp` déjà documentés plus haut, où cinq des huit runs obtenaient un verdict « Bon » plutôt que « Partiel ».

En classification, à l'inverse, Gemma3-1B-IT obtient un score parfait de 6/6, y compris sur le texte ironique déjà identifié comme cas piège du protocole, un résultat comparable au meilleur score `llama.cpp` (Galaxy S26 Ultra, 6/6 sur ses deux runs). Deux réserves nuancent toutefois ce résultat. D'une part, la confiance rapportée reste figée à 60 % sur les six réponses, quel que soit le texte, ce qui suggère une heuristique non calibrée plutôt qu'un score réellement différencié, contrairement aux runs `llama.cpp`, où la confiance variait de 2 % à 80 % selon les cas. D'autre part, les deux justifications associées aux textes classés NÉGATIF (l'écran cassé et le texte ironique) s'ouvrent systématiquement par une affirmation contradictoire au label produit lui-même (« l'utilisation de [l'élément négatif] est un atout majeur pour un smartphone ») avant de conclure correctement au sentiment négatif. Il s'agit d'un défaut de cohérence interne entre justification et label du même ordre que celui déjà observé sur les runs `llama.cpp` de l'A26 et de l'A71 (section 3.8.2), mais ici sur un framework et un modèle indépendants de ce prototype. Contrairement au run S26 Ultra en `llama.cpp`, qui identifiait explicitement un « ton sarcastique » sur le texte piégeux, cette réponse atteint le bon label sans jamais reconnaître l'ironie du texte.

Prises ensemble, ces trois vérifications croisées (raisonnement, résumé, classification) dressent un bilan nuancé plutôt qu'un verdict univoque : Gemma3-1B-IT égale ou approche `llama.cpp`/LLaMA 3.2 1B sur le raisonnement (échec commun) et la classification (score identique), mais reste net en retrait sur le résumé, avec en prime un mode de dégradation spécifique (la confiance non calibrée et les justifications internement contradictoires sur les cas négatifs) non observé sous cette forme dans les runs `llama.cpp` du même prototype.

3.8.3 Limites pratiques observées

Cinq limitations pratiques, observées directement lors de la validation du prototype (Annexe E) plutôt qu'estimées, méritent d'être détaillées ici.

La première concerne la fenêtre de contexte. Contrairement à une hypothèse initiale de fenêtre glissante (un mécanisme qui retirerait progressivement les messages les plus anciens à l'approche de la limite), le prototype ne met en œuvre aucune stratégie de gestion de la saturation du contexte : lorsque la conversation accumulée dépasse la limite de 2048 tokens fixée au lancement (`-c 2048`), `llama-cli` refuse purement et simplement de traiter la requête suivante, avec une erreur explicite et bloquante. Ce comportement a été confirmé empiriquement sur le Galaxy S26 Ultra : après environ treize échanges (douze questions du protocole suivies d'une tentative de résumé dans la même session), la requête a atteint 2226 tokens, dépassant la limite de 2048, avec l'erreur `request (2226 tokens) exceeds the available context size (2048 tokens), try increasing it`, interrompant immédiatement la session en cours. Sur l'Infinix, la même erreur bloquante survient encore plus tôt, dès la douzième question (2117 tokens), un écart probablement lié à la cinquième limitation détaillée plus bas plutôt qu'à une différence de fenêtre de contexte elle-même, identique sur les cinq appareils (`-c 2048`, fixé dans le code). Sur l'A26, l'A71 et l'A73, en revanche, la session complète (douze questions, résumé, six classifications) s'est déroulée sans jamais atteindre cette limite, alors même que ces trois appareils sont eux aussi affectés par l'incident de build décrit dans la cinquième limitation. Cela confirme, sur trois appareils supplémentaires, que le lien entre cet incident et la saturation précoce du contexte n'est pas systématique, et dépend vraisemblablement de la longueur effective des réponses générées à chaque tour plutôt que du seul incident de build en lui-même. Le prototype ne tronque pas l'historique, ne prévient pas l'utilisateur à l'avance et ne propose aucune reprise automatique. Cette absence de dégradation gracieuse constitue une limite réelle du prototype dans son état actuel, plutôt qu'une caractéristique acceptable masquée par une hypothèse de fenêtre glissante qui n'a en réalité jamais été implémentée.

La deuxième limitation concerne le temps de chargement, dont la nature diffère de celle initialement attendue. Parce que `chatbot.py` invoque un nouveau sous-processus `llama-cli` à chaque tour de parole plutôt que de charger le modèle une seule fois au démarrage (section 3.3.1), ce délai n'est pas un coût ponctuel payé une fois en début de session, mais un coût récurrent payé à chaque échange. Son existence est explicitement signalée à l'utilisateur par le prototype lui-même au lancement du mode chat (« chaque tour de parole recharge le modèle [...] un délai de quelques secondes avant la première réponse est donc normal »), et reste cohérent avec les temps de chargement de quelques secondes déjà mesurés au chapitre 2 pour ce même modèle et cette même quantification. Ce délai n'a pas été chronométré séparément lors des validations initiales

sur le S26 Ultra, l'A16, l'Infinix et l'A26, une limite méthodologique assumée pour ces quatre appareils à ce stade du protocole. Un champ `load_time_s` a depuis été ajouté à la structure `InferenceMetrics` (section 3.6.1) pour l'estimer automatiquement à chaque tour, sans chronométrage manuel : il est calculé comme la différence entre le délai total avant l'apparition du premier caractère de la réponse et le temps de prefill déjà mesuré précisément par `llama-cli`, soit `load_time_s` $\approx$ (délai avant 1er caractère) - `prefill_time_s`. Cette estimation reste approximative (elle inclut aussi le coût de démarrage du sous-processus lui-même, marginal en comparaison du chargement du modèle). Elle a ensuite été vérifiée a posteriori sur deux des quatre appareils initialement non chronométrés, via un rejeu ciblé des sept questions factuelles simples : sur le Galaxy S26 Ultra, `load_time_s` varie entre environ 1,0 et 1,5 seconde sur l'ensemble des tours mesurés ; sur le Galaxy A26, entre environ 2,8 et 5,2 secondes (le tour le plus long correspondant au premier échange de la session, cohérent avec un chargement à froid). Cet écart, plausible compte tenu du positionnement respectif des deux appareils (le S26 Ultra étant le modèle haut de gamme du corpus, l'A26 un modèle milieu de gamme), confirme la cohérence de la méthode d'estimation sur du matériel de gammes différentes. Le Galaxy A73, testé plus tard avec cette même méthode déjà intégrée nativement au prototype, confirme cette cohérence sur un troisième appareil : `load_time_s` y varie entre environ 3,1 et 5,3 secondes, du même ordre que l'A26 et cohérent avec son positionnement milieu de gamme. L'A16 et l'Infinix restent, eux, sans cette mesure.

La troisième limitation concerne la journalisation automatique des métriques, un défaut identifié puis corrigé en cours de PIR. Sur les runs 1 et 2 du Galaxy S26 Ultra ainsi que sur l'A16, la fonction `save_metrics()` n'enregistrait pas les métriques de chaque échange comme prévu, mais seulement deux entrées aberrantes générées à la fermeture de session, avec un nombre de tokens générés égal à 1 et des temps de decode mesurant en réalité le temps d'attente écoulé plutôt qu'une génération réelle (127 secondes puis 62 810 secondes, soit 17,4 heures, dans le journal de validation) ; les résultats de ces runs reposent en conséquence sur une lecture manuelle des sorties affichées à l'écran plutôt que sur les fichiers `results/metrics.json`. La cause combinait un bug de parsing des statistiques de `llama-cli` et un problème distinct de sous-processus (absence de `stdin` explicite, sortie TTY non redirigée), corrigé après le run A16 et vérifié sur un run 3 dédié du S26 Ultra, où la journalisation automatique a fonctionné correctement sur les cinq échanges testés. Les runs Infinix, A26 et A71 confirment ce correctif à plus grande échelle : les dix-huit interactions de la session Infinix (sept questions factuelles, cinq de raisonnement, un résumé, six classifications) ont toutes été journalisées correctement dans `results/metrics.json`, y compris l'échec par saturation du contexte, enregistré comme tel plutôt que comme une mesure aberrante ; sur l'A26, les vingt-trois interactions de la session (y compris la question de raisonnement posée deux fois par erreur) ont également été journalisées sans exception jusqu'aux six classifications finales ; sur l'A71,

la journalisation s’est poursuivie sans interruption à travers deux redémarrages successifs du programme (session Q/R + résumé, puis résumé de contrôle, puis classification), le compteur d’entrées progressant de façon continue d’une session à l’autre, confirmant que le correctif tient sur un quatrième appareil consécutif et reste stable même à travers plusieurs relances du programme ; sur l’A73, les dix-neuf interactions de la session (sept questions factuelles, cinq de raisonnement, un résumé, six classifications) ont de nouveau été journalisées sans exception, confirmant le correctif sur un cinquième appareil consécutif, y compris sur un appareil entièrement reconfiguré depuis zéro (Termux réinstallé, llama.cpp recompilé) plutôt que réutilisant un environnement déjà en place.

Une quatrième observation, de nature plus positive, concerne la stabilité du sous-processus : aucun crash ni fuite mémoire visible du sous-processus llama-cli n’a été observé au cours des sessions de validation, malgré le rechargement répété du modèle à chaque tour de parole. C’est un indice que l’architecture par sous-processus indépendants reste robuste sur une session d’usage prolongée, même si elle n’est pas optimale en termes de latence. Cette robustesse s’étend au cas d’échec : sur l’Infinix, la saturation du contexte à la question 12 a été gérée proprement par le prototype (échec signalé explicitement, aucune métrique polluée enregistrée), plutôt que de provoquer un plantage ou de fausser silencieusement les données comme l’aurait fait l’ancienne version de save_metrics() ; sur l’A26, l’A71 et l’A73, où cette saturation n’a jamais été atteinte, aucune instabilité alternative n’a par ailleurs été observée malgré des sessions plus longues et, pour l’A71, deux redémarrages successifs du programme (vingt-trois interactions sans interruption sur l’A26 ; sur l’A71, quatorze interactions dans la première session, puis une session de résumé de contrôle et une session de classification, sans qu’aucun redémarrage n’ait provoqué d’incident ; sur l’A73, dix-neuf interactions consécutives sans interruption dans une session unique).

Enfin, une cinquième limitation, spécifique aux appareils testés plutôt qu’au prototype lui-même, a été observée sur l’Infinix puis, de façon distincte mais avec le même symptôme, sur l’A26 et sur l’A71 : le build llama-cli installé sur l’Infinix (b9-13e6738, plus récent que les builds compilés sur le S26 Ultra et l’A16) refuse le flag --no-conversation utilisé par chatbot.py et retombe sur son propre mode conversation interne, ce qui produit, à partir du quatrième tour de la session de chat, un rappel intégral de l’historique de conversation dans chaque nouvelle réponse plutôt qu’une réponse ciblée à la seule question posée ; les builds installés sur l’A26 (b4-0eca4d4) et sur l’A71 (b1-b820cc8), deux hash de build supplémentaires distincts entre eux et de celui de l’Infinix, présentent exactement le même refus du flag et le même symptôme de rappel intégral, ce dernier se manifestant sur l’A71 par des réponses parfois décalées d’un tour entier (la réponse affichée pour une question correspondant en réalité à la question précédente). Cet incident explique vraisemblablement à la fois la saturation plus précoce du contexte observée sur l’Infinix (paragraphe ci-dessus) et une partie de la dégradation de qualité observée sur les trois

appareils (sections 3.8.1 et 3.8.2), notamment l’effondrement du score de classification sur l’Infinix et l’A71, et les contradictions internes raisonnement/label observées sur l’A26 et, dans une moindre mesure, sur l’A71. Il ne produit toutefois pas un effet identique d’un appareil à l’autre, puisque ni l’A26 ni l’A71 n’ont connu de saturation du contexte malgré le même incident. Le cas de l’A71 est particulièrement instructif à cet égard : `llama.cpp` y avait déjà été compilé antérieurement pour les besoins du chapitre 2, avant même le clonage du dépôt complet `llm-smartphone` réalisé pour ce test. Cela indique que l’incident ne se limite pas à une fenêtre de clonage récente et concerne une plage de versions de `llama-cli` plus large qu’initialement supposé sur les seuls cas Infinix/A26. Cette récurrence sur trois appareils consécutifs, avec trois hash de build différents, renforce l’hypothèse que l’incident est lié à la version de `llama.cpp/llama-cli` disponible au moment de la compilation plutôt qu’à une particularité isolée d’un seul appareil. Il illustre une limite de reproductibilité plus générale que la seule variabilité du modèle déjà documentée pour l’A16 : compiler ou cloner le même dépôt à des dates différentes, sur des appareils différents, expose le protocole de test à des versions différentes de ses propres dépendances, une variable non contrôlée susceptible de produire des écarts de résultats qui, sans cette vérification, auraient pu être attribués à tort au matériel testé plutôt qu’au logiciel.

Le test du Galaxy A73, sixième et dernier appareil du corpus, apporte une quatrième occurrence de cet incident, mais avec une variante inédite et plus sévère. Le build installé sur cet appareil (`b10739-d08c7872d`), nettement postérieur aux trois précédents, ne se contente plus d’ignorer silencieusement le flag `--no-conversation` en retombant sur son propre mode conversation : il le rejette purement et simplement comme argument invalide (`invalid argument: -no-cnv`), provoquant un échec bloquant et empêchant toute génération. Contrairement aux trois occurrences précédentes, un simple constat empirique ne suffisait donc plus : une correction du code source de `chatbot.py` a été nécessaire (retrait du flag `-no-cnv`, le flag `-st/--single-turn` restant seul suffisant selon la documentation `--help` de ce build), déployée puis vérifiée fonctionnelle sur cet appareil. Une fois ce correctif appliqué, le même symptôme de rappel intégral de l’historique que sur les trois builds précédents réapparaît malgré tout, ce qui confirme qu’il s’agit bien du même mode conversation interne de `llama-cli`, simplement déclenché différemment selon la version du binaire. Cette quatrième occurrence, sur un build sensiblement plus récent que les trois précédents et strictement plus récent que celui du S26 Ultra (`b10154-0e4a03622`, qui n’avait pourtant pas rejeté `-no-cnv` comme argument invalide), élargit encore la plage de versions de `llama.cpp/llama-cli` concernées par cet incident, et illustre qu’une même cause logicielle peut produire des symptômes de sévérité croissante selon la version exacte du binaire, allant d’un simple repli silencieux à un échec bloquant nécessitant une intervention corrective directe sur le code du prototype.

3.9 Documentation du pipeline complet

Pour synthétiser l'ensemble des étapes décrites dans ce chapitre, le déroulement complet du pipeline applicatif, depuis le fichier modèle stocké sur l'appareil jusqu'à l'enregistrement des métriques, peut être résumé comme suit :

1. La disponibilité du binaire `llama-cli` et du fichier GGUF est vérifiée par `load_backend()`, sans chargement effectif du modèle à ce stade (section 3.3.1).
2. À chaque tour de parole, un nouveau sous-processus `llama-cli` est lancé via `subprocess.Popen`, qui charge alors le modèle depuis le stockage de l'appareil et exécute l'inférence demandée.
3. La saisie de l'utilisateur est transformée en prompt structuré par `build_chat_prompt()`, qui y intègre l'historique de conversation et l'instruction système appropriée à la tâche en cours.
4. L'inférence s'exécute entièrement sur CPU ARM64, avec les optimisations vectorielles NEON héritées de la compilation de `llama.cpp` (chapitre 2, section 1.1.3) et une parallélisation sur plusieurs threads.
5. Les tokens générés sont transmis en flux continu vers la console, où ils s'affichent progressivement grâce à la bibliothèque `rich`.
6. Une fois la génération achevée, l'objet `InferenceMetrics` est calculé puis, en principe, enregistré de façon incrémentale dans `results/metrics.json` via `save_metrics()` (mécanisme dont le dysfonctionnement en mode interactif est discuté en section 3.8.3).
7. L'historique de conversation est mis à jour avec le nouvel échange, prêt à être réintégré dans le prompt du tour de parole suivant.

Quatre propriétés se dégagent de ce pipeline, qui en résument l'intérêt pour la démonstration recherchée dans ce chapitre. Le traitement est intégralement local ("stateless côté serveur") : à aucune étape une communication réseau n'est nécessaire, ce qui confirme concrètement la propriété de confidentialité et de disponibilité hors-ligne mise en avant dans l'état de l'art (chapitre 1). La persistance des métriques reste légère, sous forme de fichiers JSON locaux, sans dépendance à une base de données externe. Le pipeline est également conçu pour être extensible : l'ajout d'une nouvelle tâche ne nécessite qu'une nouvelle entrée dans `TASK_PROMPTS` et une fonction de mode dédiée, sans modification du cœur du pipeline d'inférence. Enfin, le mode `--mock` permet de valider l'ensemble de cette chaîne applicative indépendamment de la disponibilité d'un modèle réel, ce qui a facilité le développement itératif du prototype.

3.10 Perspectives d'extension

Le prototype CLI présenté dans ce chapitre constitue une base fonctionnelle et testée, mais volontairement limitée dans son périmètre, à partir de laquelle plusieurs directions d'extension peuvent être envisagées pour des travaux futurs.

Une extension Android native est la direction la plus immédiate : le ViewModel Kotlin déjà présenté au chapitre 2 (section 2.3.1, et détaillé en Annexe C du chapitre 2) pourrait être directement connecté à une API HTTP légère (Flask ou FastAPI exécutée dans Termux) exposant le binaire `llama-cli` déjà validé dans ce chapitre, une approche cohérente avec le choix architectural retenu pour `chatbot.py` (section 3.2.1), ou au binding Android natif de `llama.cpp`, ce qui permettrait de combiner l'interface graphique déjà développée pour ML Kit GenAI avec le moteur d'inférence `llama.cpp` validé dans ce chapitre.

L'intégration d'une composante de génération augmentée par récupération (RAG, Retrieval-Augmented Generation) constitue une deuxième direction : une base vectorielle légère (ChromaDB ou FAISS, toutes deux exécutables localement) permettrait au prototype de répondre à des questions portant sur un corpus de documents personnels de l'utilisateur, dans les limites de la RAM disponible sur l'appareil, ce qui répondrait à une limitation actuelle du prototype (absence de connaissance au-delà de ce qui est contenu dans les poids du modèle).

Le remplacement du modèle de base par une variante affinée via QLoRA (chapitre 1, section 1.1) sur des données spécifiques à un domaine métier (service client, assistance de premier niveau, FAQ multilingue) constitue une troisième piste, la procédure de quantification post-entraînement vers le format GGUF étant déjà documentée dans l'état de l'art de ce PIR.

Enfin, sur le plan strictement ergonomique, le remplacement de l'interface en ligne de commande par une interface graphique légère (Gradio ou Streamlit), exécutable localement via Termux, permettrait de rendre le prototype accessible à des utilisateurs non familiers avec un terminal, pour des démonstrations ou des tests d'utilisabilité plus larges que ceux réalisés dans le cadre de ce PIR.